Week 4 — Data Structures & Loops

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Lists and tuples
- Dictionaries and sets
- for and while loops
- Looping over structured data

Apply — ~30 min

- Code-along: list operations
- Code-along: loop over a list of dicts
- Code-along: student roster analyzer

Working With Collections

Real programs don't work with a single value — they work with **collections**: a list of users, a dictionary of settings, a set of tags.

This week you'll learn Python's four core data structures and how to loop through them.

Lists

An ordered, mutable sequence. Access by index.

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])        # apple
print(fruits[-1])       # cherry
print(fruits[1:3])      # ['banana', 'cherry']

fruits.append("mango")
fruits.remove("banana")
```

Tuples

An ordered, **immutable** sequence — use when the data shouldn't change.

```
coordinates = (40.7128, -74.0060)
rgb = (255, 128, 0)

print(coordinates[0])   # 40.7128
```

Dictionaries

Key-value pairs. Access by key, not by position.

```
student = {
    "name": "Sara",
    "score": 91,
    "subject": "Python"
}

print(student["name"])
print(student.get("grade", "N/A"))
```

Check for Understanding #1

Given this dictionary, which line raises an error?

```
person = {"name": "Alex", "age": 30}
```

A) `person["name"]`
B) `person["email"]`
C) `person.get("email")`
D) `person.get("email", "N/A")`

Sets

An unordered collection of **unique** values.

```
a = {"python", "java", "javascript"}
b = {"python", "rust", "go"}

print(a & b)    # intersection: {'python'}
print(a | b)    # union: all unique values
print(a - b)    # difference: only in a
```

for Loops

Iterate over any sequence with a `for` loop.

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

for i in range(5):
    print(i)    # 0, 1, 2, 3, 4
```

while Loops and break / continue

`while` runs as long as a condition is `True`.

```
count = 0
while count < 3:
    print(count)
    count += 1

for n in range(10):
    if n == 5:
        break        # stop the loop
    if n % 2 == 0:
        continue     # skip to next iteration
    print(n)
```

Check for Understanding #2

What does this loop print?

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

A) `0 1 2 3 4`

B) `0 1 2`

C) `0 1 2 3`

D) `1 2 3 4`

Looping Over Structured Data

Combine loops and dictionaries to work with real data.

```
students = [
    {"name": "Sara",   "score": 91},
    {"name": "Marcus", "score": 83},
    {"name": "Priya",  "score": 95},
]

for student in students:
    print(student["name"], student["score"])
```

Check for Understanding #3

How do you loop over both keys **and** values of a dictionary?

```
scores = {"Sara": 91, "Marcus": 83}
```

A) `for k in scores:`

B) `for v in scores.values():`

C) `for k, v in scores.items():`

D) `for k, v in scores:`

Time to Apply

Let's put data structures and loops to work.

Mentor 2 takes over

Assignment 4 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — list indexing and slicing
- `warmup2.py` — dictionary operations with `.items()`
- `warmup3.py` — set union, intersection, difference
- `warmup4.py` (in assignment) — loop practice

Part 2 — Student Roster Analyzer

- Loop through a list of student dicts
- Find top scorer, calculate average, collect unique subjects, list high scorers
- Data file provided in `week-4/data/roster.py`

Code-Along 1: List Operations

Access list items without a loop.

```
numbers = [42, 14, 78, 83, 5, 61, 29, 40]

print("First:   ", numbers[0])
print("Last:    ", numbers[-1])
print("Middle:  ", numbers[2:6])
print("Reversed:", numbers[::-1])
```

Code-Along 2: Loop Over a List of Dicts

Build toward the roster analyzer pattern.

```
students = [
    {"name": "Jazmine", "score": 88, "subject": "Python"},
    {"name": "Luis",    "score": 74, "subject": "Data"},
    {"name": "Sara",    "score": 91, "subject": "Python"},
]

for s in students:
    print(f"{s['name']}: {s['score']}")
```

Code-Along 3: Roster Analyzer — Top Scorer

Find the highest score without using `max()`.

```
top_name = ""
top_score = 0

for student in students:
    if student["score"] > top_score:
        top_score = student["score"]
        top_name = student["name"]

print(f"Top scorer: {top_name} ({top_score})")
```

Extension Challenge

Ungraded extension: After finding the top scorer, add a second loop that calculates the class average.

Then: use a **set** to collect all unique subjects as you loop through students. Print the set at the end.

These two additions complete the full mini-project output.

Wrap-Up

Today you learned:

- Lists, tuples, dictionaries, and sets — when to use each
- for loops, while loops, break, and continue
- Looping over a list of dictionaries

This week: Complete warmups 1–3 and the Roster Analyzer. Submit via pull request.

Next week: Loop patterns, list comprehensions, and algorithms — sharpening iteration into something more precise.